

Informe técnico: Sistema de Reserva de Hoteles

Grupo: Grupo 1

Integrantes:

- Andrés Encalada
- Nayeli Barbecho
- Jordy Romero
- David Villa
- Karen Ortiz

Proyecto: API REST para la gestión de reservas de hotel

1. Resumen

El presente informe describe el desarrollo de una API REST para un sistema de reserva de hoteles. La aplicación permite registrar e iniciar sesión de usuarios, administrar habitaciones, crear reservas, marcar pagos, generar facturas y consultar estadísticas administrativas.

La solución fue desarrollada con FastAPI, SQLAlchemy asíncrono y PostgreSQL. Además, cumple con el requisito de publicar los servicios con documentación mediante Swagger, disponible en la ruta [/docs](#).

2. Objetivo

El objetivo del proyecto es implementar un backend funcional para la gestión de reservas hoteleras, aplicando una arquitectura organizada por capas, autenticación mediante JWT, control de acceso por roles y documentación interactiva de los endpoints mediante Swagger UI.

3. Tecnologías utilizadas

Tecnología	Uso
Python 3.12+	Lenguaje principal del backend
FastAPI	Framework web para exponer la API REST
SQLAlchemy 2.0 async	ORM y acceso asíncrono a la base de datos
asyncpg	Driver asíncrono para PostgreSQL
PostgreSQL 15	Base de datos relacional
Docker Compose	Levantamiento local de la base de datos
Pydantic	Validación de datos y esquemas de entrada/salida
JWT	Autenticación sin estado mediante tokens
bcrypt	Hash seguro de contraseñas
uv	Gestión de dependencias y entorno de ejecución

4. Inicialización del proyecto

4.1. Requisitos

- Python 3.12 o superior.
- Docker y Docker Compose.
- **uv** instalado.

4.2. Variables de entorno

Para ejecutar el sistema se debe crear un archivo **.env** en la raíz del proyecto, tomando como base el archivo **.env.example**:

```
SECRET_KEY=clave-secreta-cambiar-en-produccion
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
DATABASE_URL=postgresql+asyncpg://hotel_user:hotel_password@localhost:5432/hotel_reservations
```

4.3. Comandos de arranque

Los siguientes comandos permiten instalar dependencias, levantar la base de datos, crear las tablas, cargar datos de prueba e iniciar el servidor:

```
uv sync
docker compose up -d
uv run app/scripts/init_db.py
uv run app/scripts/seed_users.py
uv run app/scripts/seed_demo_data.py
uv run uvicorn main:app --app-dir app --reload
```

Cuando el servidor esté iniciado, la API queda disponible en:

- API: **http://127.0.0.1:8000**
- Swagger UI: **http://127.0.0.1:8000/docs**
- ReDoc: **http://127.0.0.1:8000/redoc**
- OpenAPI JSON: **http://127.0.0.1:8000/openapi.json**

4.4. Credenciales de prueba

Rol	Email	Password
Administrador	admin@hotel.com	admin123
Huésped	huesped@hotel.com	huesped123
Administrador demo	recepcion@hotel.com	recepcion123

Rol	Email	Password
Huésped demo	maria.gomez@hotel.com	maria123
Huésped demo	carlos.perez@hotel.com	carlos123
Huésped demo	ana.torres@hotel.com	ana123

4.5. Datos demo adicionales

Se agregó el script `app/scripts/seed_demo_data.py` para cargar información de prueba más completa. El script es idempotente, por lo que puede ejecutarse varias veces sin duplicar los mismos registros demo.

El script crea o actualiza:

- 4 usuarios demo adicionales.
- 8 habitaciones de tipo `Individual`, `Doble`, `Suite` y `Familiar`.
- 6 reservas en estados `pagado`, `pendiente` y `cancelada`.
- 3 facturas asociadas a reservas pagadas.

Después de ejecutar `seed_users.py` y `seed_demo_data.py`, la base queda con:

Entidad	Cantidad
Usuarios	6
Habitaciones	8
Reservas	6
Facturas	3

5. Documentación Swagger

FastAPI genera automáticamente la especificación OpenAPI a partir de:

- Rutas declaradas con `APIRouter`.
- Modelos Pydantic usados como request y response models.
- Tags configurados en los controladores.
- Dependencia OAuth2 Bearer usada para endpoints protegidos.

Para probar los servicios protegidos en Swagger se realiza el siguiente procedimiento:

1. Abrir `http://127.0.0.1:8000/docs`.
2. Ejecutar `POST /api/auth/login`.
3. Ingresar el email en el campo `username` y la contraseña en `password`.
4. Copiar el `access_token`.
5. Presionar `Authorize`.
6. Pegar el token como Bearer token.

5.1. Evidencia visual de Swagger

Las siguientes capturas muestran que los servicios fueron publicados y probados mediante Swagger UI.

Vista general de Swagger

API de Reservas de Hotel 1.0.0 OAS 3.1

/openapi.json

API backend para gestionar reservas de hotel.

Funcionalidades principales:

- Autenticacion con JWT.
- Control de acceso por roles: `admin` y `user`.
- Gestion de usuarios, habitaciones, reservas y facturas.
- Dashboard administrativo con estadísticas generales.

Uso en Swagger:

1. Ejecutar `POST /api/auth/login` con las credenciales de prueba.
2. Copiar el `access_token` retornado.
3. Pulsar `Authorize` y pegar el token con formato Bearer.

Authorize

Autenticación Registro e inicio de sesion. Devuelve tokens JWT para consumir endpoints protegidos.

POST	/api/auth/login	Login	▼
POST	/api/auth/registro	Registro	▼

Usuarios Consulta de perfil, cambio de password y administracion de usuarios.

GET	/api/usuarios/me	Obtener Perfil	🔒 ▼
PATCH	/api/usuarios/me/password	Cambiar Password	🔒 ▼
GET	/api/usuarios/	Listar Usuarios	🔒 ▼
DELETE	/api/usuarios/{id}	Eliminar Usuario	🔒 ▼

Habitaciones Busqueda, consulta y administracion de habitaciones del hotel.

GET	/api/habitaciones/	Listar Habitaciones	🔒 ▼
POST	/api/habitaciones/	Crear Habitacion	🔒 ▼
GET	/api/habitaciones/{id}	Obtener Habitacion	🔒 ▼
DELETE	/api/habitaciones/{id}	Eliminar Habitacion	🔒 ▼
PATCH	/api/habitaciones/{id}/estado	Cambiar Estado	🔒 ▼

Reservas Creacion, consulta, pago, notificacion y cancelacion de reservas.

GET	/api/reservas/	Listar Reservas	🔒 ▼
POST	/api/reservas/	Crear Reserva	🔒 ▼
GET	/api/reservas/mis-reservas	Listar Mis Reservas	🔒 ▼
PATCH	/api/reservas/{id}/pagar	Marcar Pagado	🔒 ▼
POST	/api/reservas/{id}/notificar-pago	Notificar Pago	🔒 ▼
PATCH	/api/reservas/{id}/cancelar	Cancelar Reserva	🔒 ▼

Facturas Consulta de facturas generadas a partir de reservas pagadas.

GET	/api/facturas/	Listar Facturas	🔒 ▼
GET	/api/facturas/{reserva_id}	Obtener Factura	🔒 ▼

Dashboard Indicadores administrativos de reservas, habitaciones e ingresos.

GET	/api/dashboard/estadisticas	Ver Estadisticas	🔒 ▼
-----	-----------------------------	------------------	-----

default

GET	/	Root	▼
-----	---	------	---

Schemas

Body_login_api_auth_login_post > Expand all object

FacturaResponse > Expand all object

HTTPValidationError > Expand all object

HabitacionCreate > Expand all object

HabitacionResponse > Expand all object

PasswordChange > Expand all object

ReservaCreate >	Expand all	object
ReservaResponse >	Expand all	object
Role >	Expand all	string
Token >	Expand all	object
UserCreate >	Expand all	object
UserResponse >	Expand all	object
ValidationError >	Expand all	object

En esta vista se observan los módulos documentados por tags: Autenticación, Usuarios, Habitaciones, Reservas, Facturas y Dashboard. Cada sección agrupa sus endpoints, métodos HTTP, parámetros y esquemas de respuesta.

Autorización con JWT

Autenticación Registro e inicio de sesion. Devuelve tokens JWT para consumir endpoints protegidos.

POST

/api/auth/login

Login

Parameters

Cancel

Reset

No parameters

Request body

required

application/x-www-form-urlencoded

grant_type

string | (string | null)

pattern: ^password\$

password

☐ Send empty value

username

string

required

admin@hotel.com

password

string(\$password)

required

.....

scope

string

scope

☒ Send empty value

client_id

string | (string | null)

string

☐ Send empty value

client_secret

string | (string | null)(\$password)

.....

☐ Send empty value

Execute

Clear

Responses

Curl

curl -X 'POST' \

'http://127.0.0.1:8000/api/auth/login' \

-H 'accept: application/json' \

-H 'Content-Type: application/x-www-form-urlencoded' \

-d 'grant_type=password&username=admin%40hotel.com&password=admin123&scope=&client_id=string&client_secret=*****'

Request URL

http://127.0.0.1:8000/api/auth/login

Server response

Code

Details

200

Response body

{

"access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWI0IjphZG1pbkBob3Rlbc5jb20iLCJyb2xlIjoieWVMTaW4iLCJleHAiOiE3ODIwIjE3MTZ9.3Yb5XuM2tKENHG0BzSyRBn9CjDsJNLHjhZw86G_mRP",

"token_type": "bearer"

}

Response headers

content-length: 198

content-type: application/json

date: Sun, 21 Jun 2026 15:58:35 GMT

server: uvicorn

Responses

Code

Description

Links

200

Successful Response

No links

Media type

application/json

Controls Accept header

Example Value

Schema

{

"access_token": "string",

"token_type": "string"

}

La ventana de autorización permite registrar el token JWT devuelto por `POST /api/auth/login`. Con esto Swagger envía el token Bearer en las peticiones protegidas.

Endpoint administrativo de estadísticas

7 / 19

Dashboard Indicadores administrativos de reservas, habitaciones e ingresos.

GET /api/dashboard/estadisticas Ver Estadísticas

Parameters

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/api/dashboard/estadisticas' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhZG1pbkBob3RlbC5jb20iLCJyb2x1Ijo1YWRTaW4iLCJleHAiOiE3ODIwNjE3MzZ9._g0bsrQiTuKp0EUBc-uL7MXeZw1uHjDb40LNSY0C7YA'
```

Request URL

```
http://127.0.0.1:8000/api/dashboard/estadisticas
```

Server response

Code	Details
200	Response body <pre>{ "reservas": { "total": 6, "pendientes": 2, "pagadas": 3, "canceladas": 1 }, "habitaciones": { "disponibles": 6, "ocupadas": 2 }, "ingresos_totales": 375 }</pre> Download

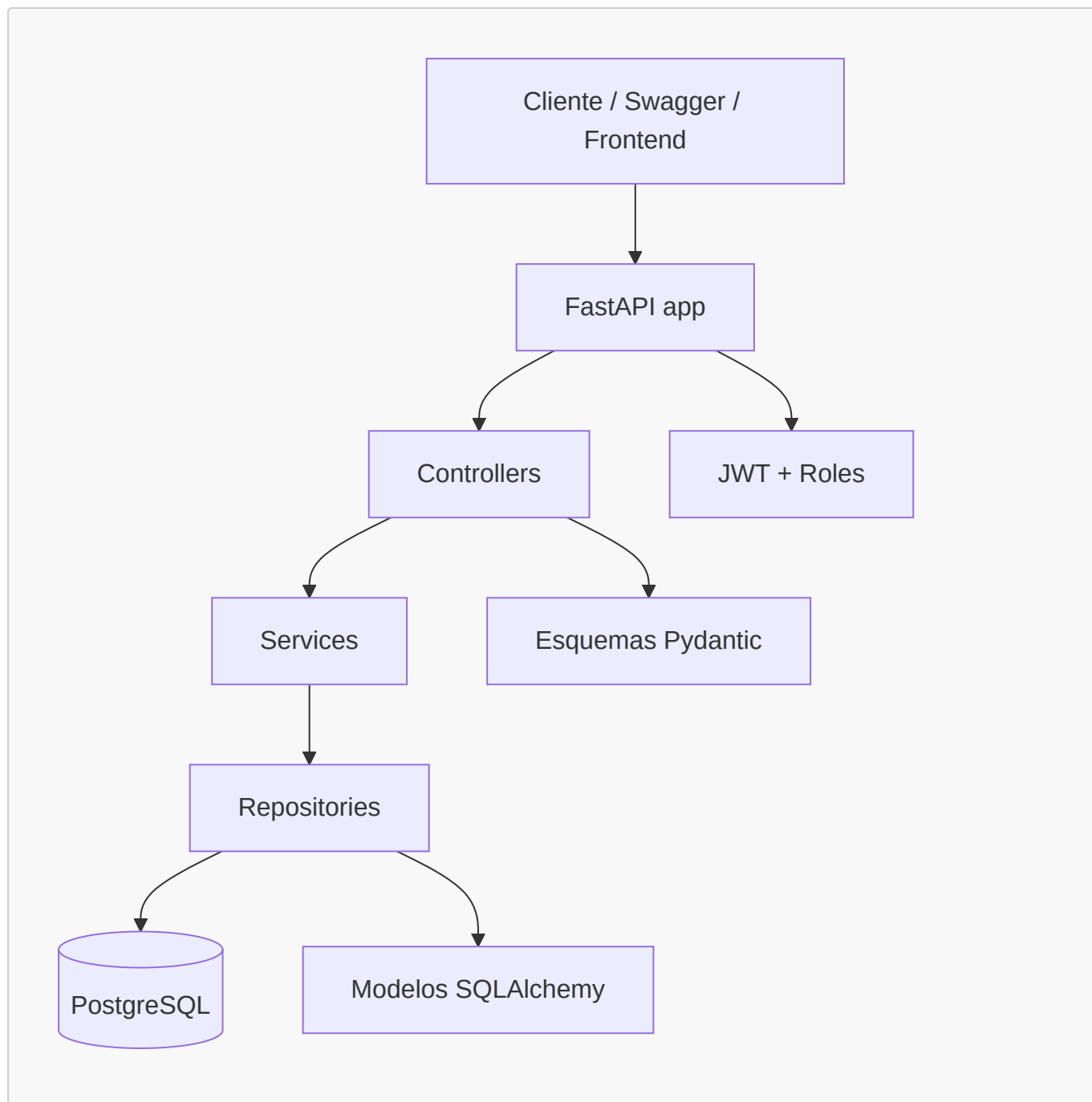
La captura evidencia el consumo de `/api/dashboard/estadisticas`, endpoint protegido para administradores. Este servicio devuelve totales de reservas, habitaciones disponibles/ocupadas e ingresos calculados desde la base de datos.

6. Estructura del proyecto

```
.
├── app
│   ├── controllers    # Endpoints HTTP y dependencias de FastAPI
│   └── core           # Configuración, seguridad y conexión a base de
datos
│   ├── models         # Modelos SQLAlchemy y esquemas Pydantic
│   ├── repositories   # Consultas y persistencia de datos
│   ├── scripts        # Inicialización y datos de prueba
│   ├── services       # Reglas de negocio
│   └── main.py        # Punto de entrada de FastAPI
├── docs
├── img                # Capturas de Swagger para evidencias del informe
├── docker-compose.yml # Servicio PostgreSQL
├── pyproject.toml     # Dependencias del proyecto
├── README.md          # Guía rápida
└── INFORME.md         # Informe técnico del sistema
```

7. Arquitectura

El sistema sigue una arquitectura por capas similar a Modelo-Repositorio-Servicio-Controlador.



Responsabilidades principales:

- **controllers**: reciben la petición HTTP, validan dependencias y devuelven respuestas.
- **services**: concentran reglas de negocio, como pagos, cancelaciones y cálculos.
- **repositories**: encapsulan consultas SQLAlchemy.
- **models**: definen tablas de base de datos y esquemas de entrada/salida.
- **core**: contiene configuración, seguridad y sesión de base de datos.

8. Módulos implementados

8.1. Autenticación

Permite registrar usuarios e iniciar sesión. El login retorna un JWT con el email y rol del usuario.

Endpoints:

Método	Ruta	Acceso	Descripción
POST	/api/auth/login	Público	Autentica usuario y devuelve token
POST	/api/auth/registro	Público	Registra un usuario nuevo

8.2. Usuarios

Permite consultar el perfil autenticado, cambiar contraseña y administrar usuarios desde el rol administrador.

Método	Ruta	Acceso	Descripción
GET	/api/usuarios/me	Usuario autenticado	Obtiene perfil propio
PATCH	/api/usuarios/me/password	Usuario autenticado	Cambia contraseña
GET	/api/usuarios/	Admin	Lista usuarios
DELETE	/api/usuarios/{id}	Admin	Elimina usuario

8.3. Habitaciones

Permite consultar habitaciones y administrarlas desde el rol administrador.

Método	Ruta	Acceso	Descripción
GET	/api/habitaciones/	Usuario autenticado	Lista habitaciones con filtros
GET	/api/habitaciones/{id}	Usuario autenticado	Obtiene habitación por id
POST	/api/habitaciones/	Admin	Crea habitación
PATCH	/api/habitaciones/{id}/estado	Admin	Cambia disponibilidad
DELETE	/api/habitaciones/{id}	Admin	Elimina habitación

8.4. Reservas

Permite crear reservas, consultar reservas propias, administrar pagos y cancelar reservas.

Método	Ruta	Acceso	Descripción
POST	/api/reservas/	Usuario autenticado	Crea reserva
GET	/api/reservas/	Admin	Lista todas las reservas
GET	/api/reservas/mis-reservas	Usuario autenticado	Lista reservas propias
PATCH	/api/reservas/{id}/pagar	Admin	Marca una reserva como pagada y genera factura
POST	/api/reservas/{id}/notificar-pago	Usuario autenticado	Simula notificación de pago pendiente

Método	Ruta	Acceso	Descripción
PATCH	/api/reservas/{id}/cancelar	Usuario autenticado	Cancela reserva

8.5. Facturas

Permite consultar facturas generadas por reservas pagadas.

Método	Ruta	Acceso	Descripción
GET	/api/facturas/	Admin	Lista todas las facturas
GET	/api/facturas/{reserva_id}	Usuario autenticado	Obtiene factura por reserva

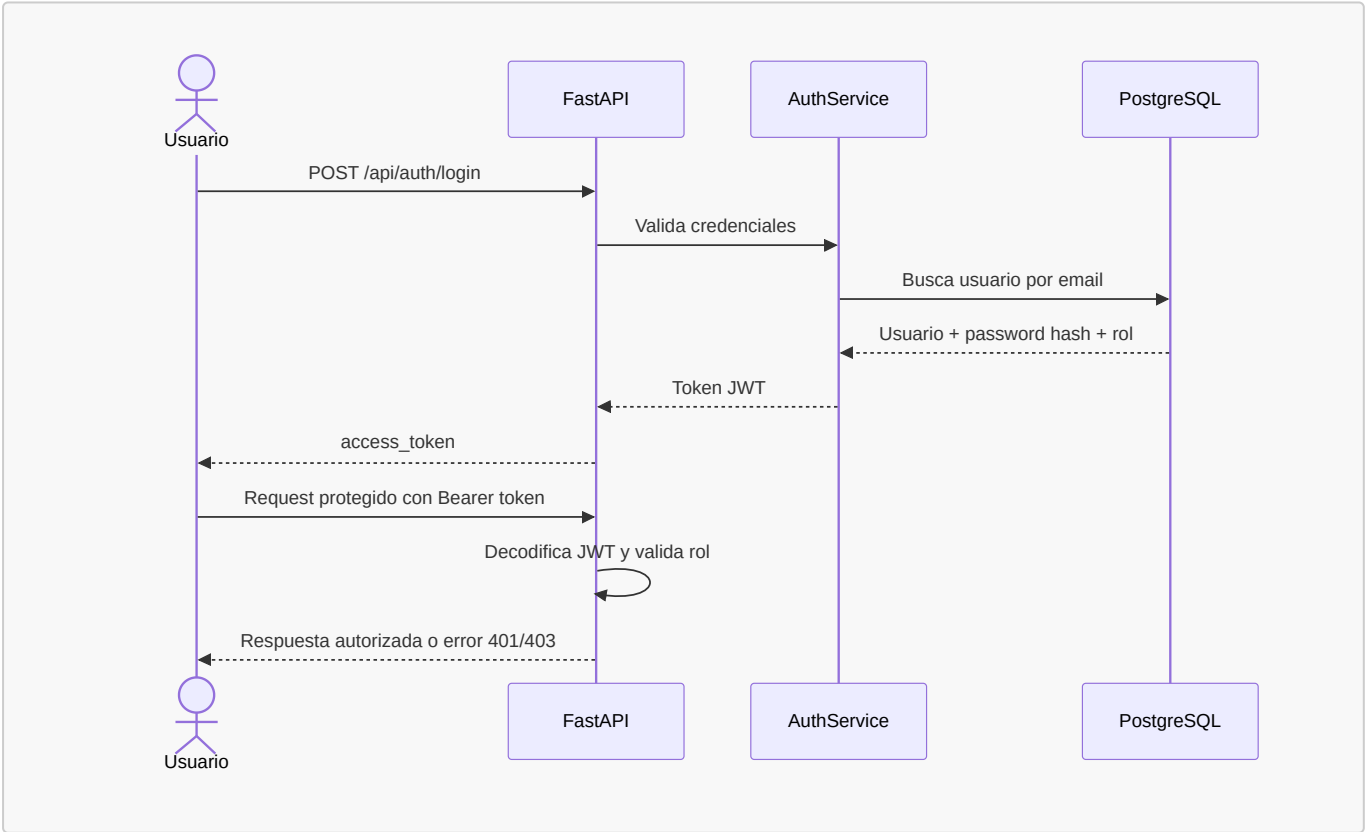
8.6. Dashboard

Módulo administrativo para consultar indicadores generales.

Método	Ruta	Acceso	Descripción
GET	/api/dashboard/estadisticas	Admin	Muestra estadísticas de reservas, habitaciones e ingresos

9. Seguridad

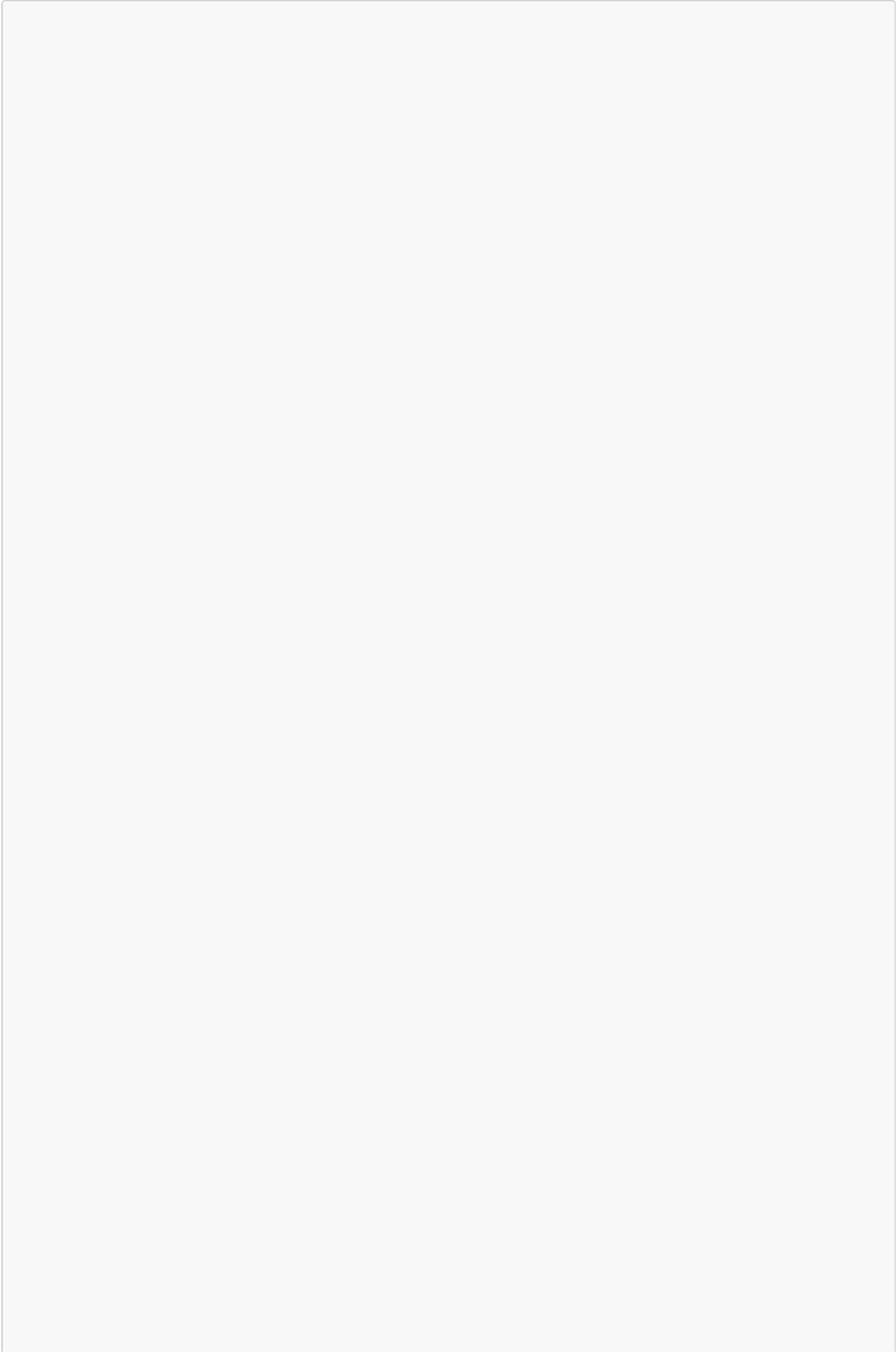
La seguridad se basa en JWT y control de acceso por roles.

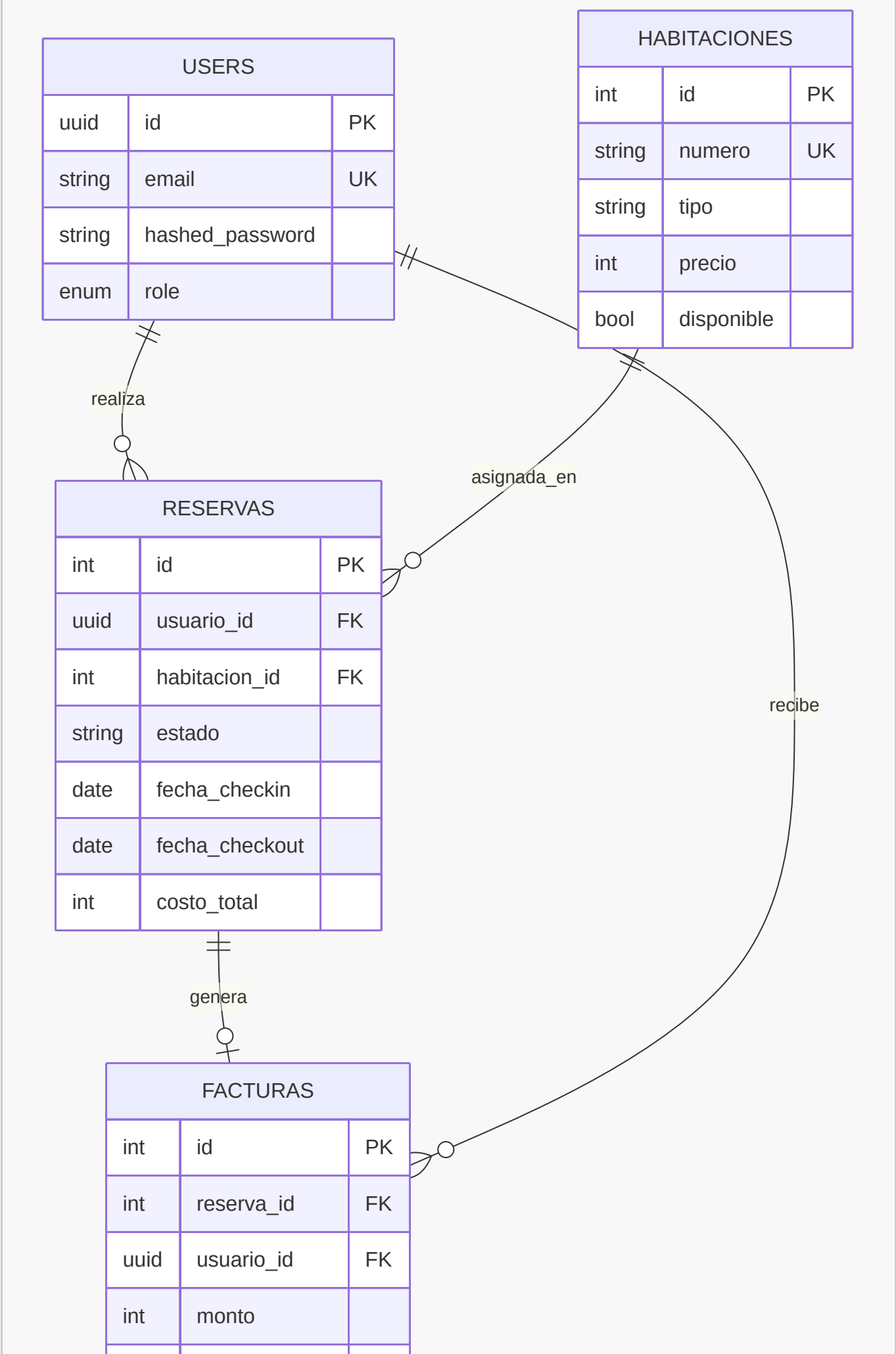


Reglas aplicadas:

- Endpoints públicos: login y registro.
- Endpoints autenticados: perfil, habitaciones, reservas propias, factura por reserva.
- Endpoints administrativos: administración de habitaciones, usuarios, reservas, facturas y dashboard.

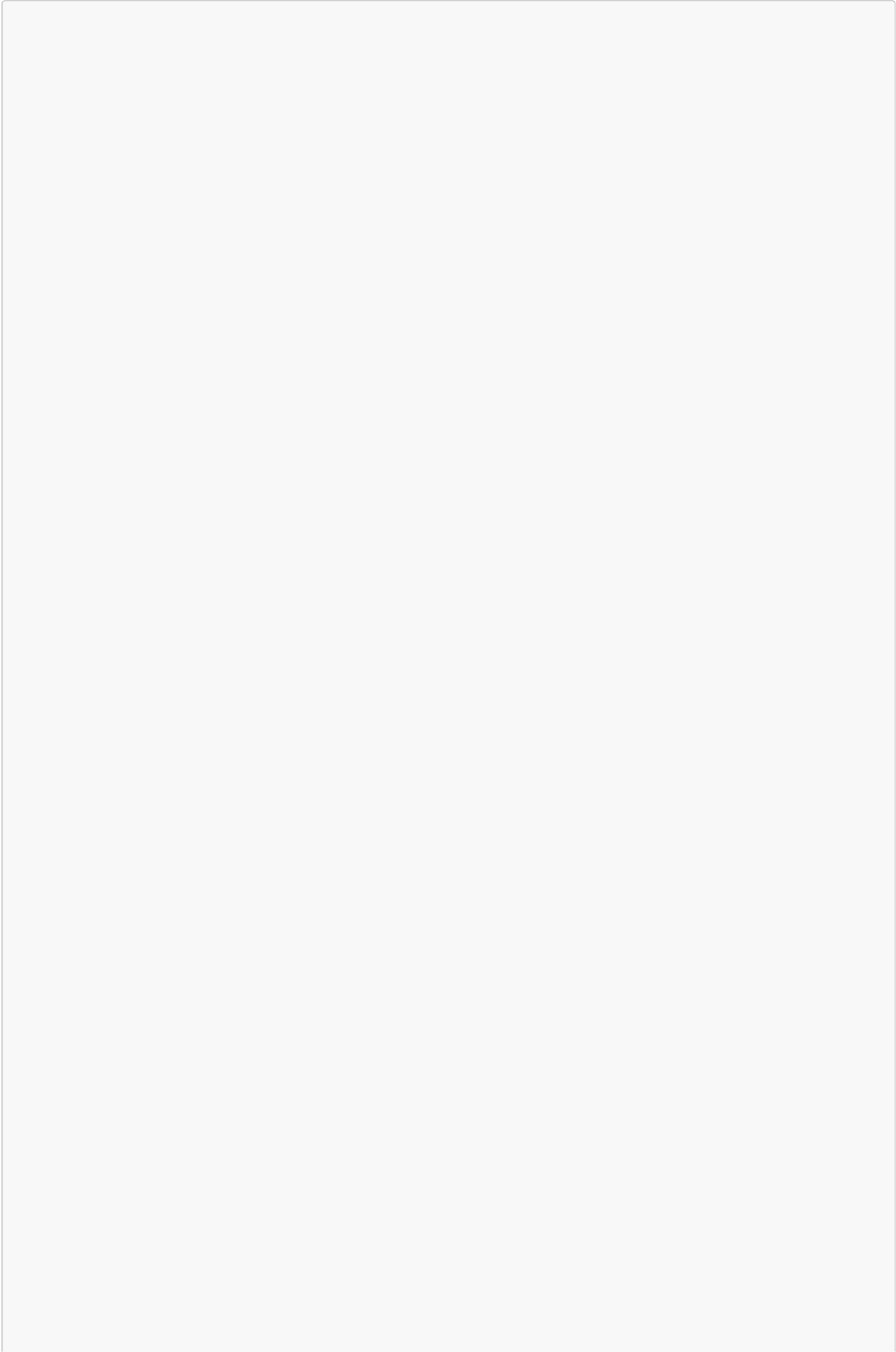
10. Modelo de datos

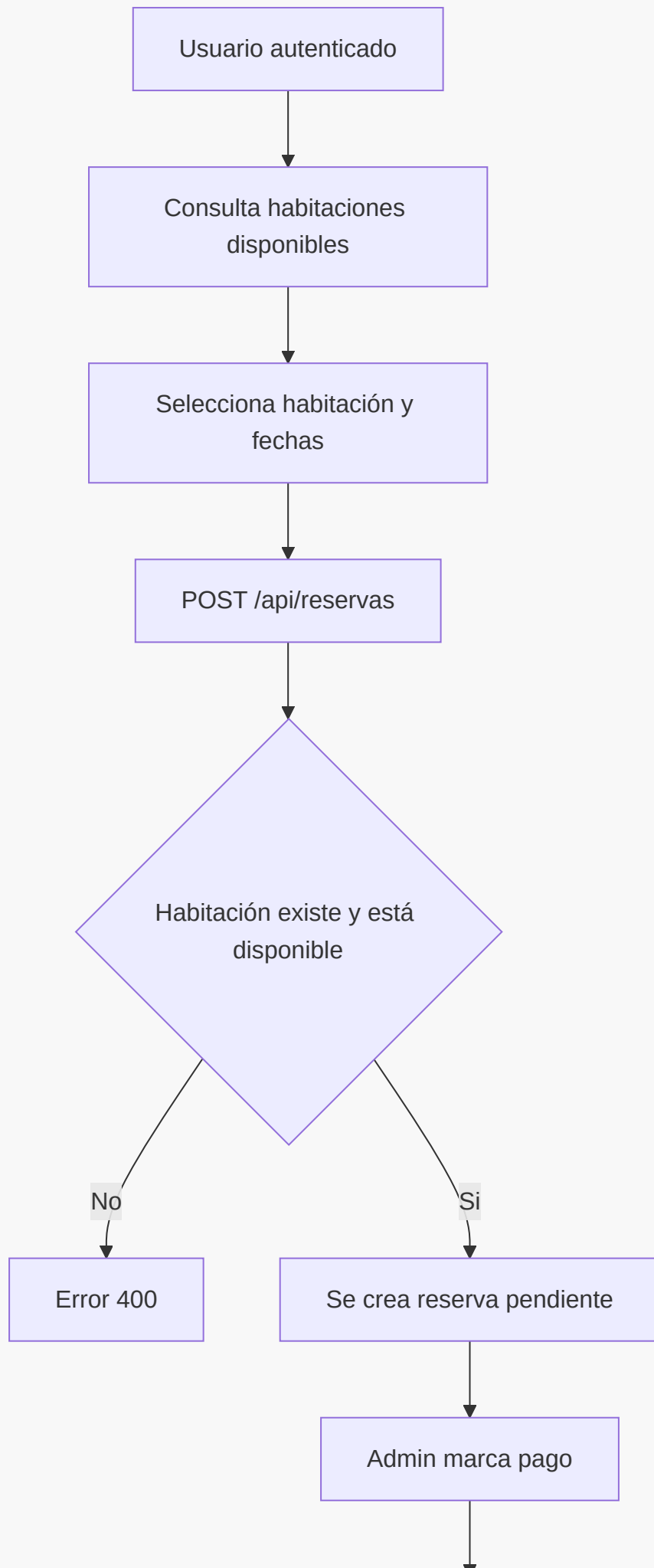


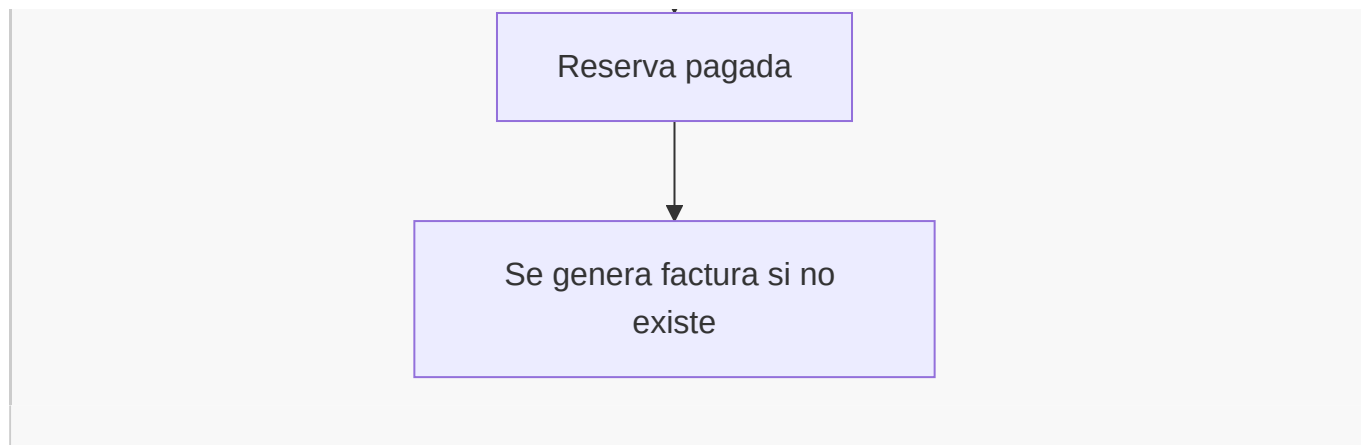


	date	fecha_emision	
--	------	---------------	--

11. Flujo principal de reserva







12. Reglas de negocio relevantes

- La fecha de salida debe ser posterior a la fecha de ingreso.
- Solo usuarios autenticados pueden consultar habitaciones y crear reservas.
- Solo administradores pueden crear, eliminar o cambiar disponibilidad de habitaciones.
- Solo administradores pueden listar todas las reservas y marcar pagos.
- Al marcar una reserva como pagada, se genera una factura si todavía no existe.
- El dashboard calcula totales de reservas, estados, habitaciones disponibles/ocupadas e ingresos.

13. Scripts de soporte

Script	Descripción
<code>app/scripts/init_db.py</code>	Crea las tablas declaradas con SQLAlchemy en PostgreSQL
<code>app/scripts/seed_users.py</code>	Crea usuarios básicos de prueba: admin y huésped
<code>app/scripts/seed_demo_data.py</code>	Carga datos demo completos: usuarios, habitaciones, reservas y facturas

14. Endpoints de soporte y documentación

Ruta	Descripción
<code>/</code>	Mensaje de bienvenida
<code>/docs</code>	Swagger UI
<code>/redoc</code>	Documentación ReDoc
<code>/openapi.json</code>	Especificación OpenAPI

15. Conclusiones

El proyecto cumple con la publicación de servicios documentados mediante Swagger. La API está organizada por módulos, utiliza validación con Pydantic y separa responsabilidades en capas, lo cual facilita el mantenimiento, la comprensión del código y la posible incorporación de nuevas funcionalidades.

Además, se implementó autenticación con JWT y control de acceso por roles, permitiendo diferenciar operaciones públicas, operaciones de usuarios autenticados y operaciones exclusivas para administradores.

También se prepararon datos de prueba para demostrar el funcionamiento de los módulos principales desde Swagger.

Posibles mejoras futuras:

- Agregar migraciones con Alembic.
- Agregar pruebas automatizadas.
- Restringir que un usuario solo pueda cancelar reservas propias.
- Registrar notificaciones reales por correo o mensajería.
- Agregar paginación en listados administrativos.